

Ciclo de vida de la venta e integridad del inventario

El bug de Math.max que escribía inventario falso, los dos estados terminales de una venta (confirmada / anulada), la migración V2 y los 24 tests que lo blindan

Bug corregido: cancelar una compra escribía stock 0 con Math.max en vez de fallar

Regla: si el stock de la compra ya se consumio, la compra NO se cancela (409)

Venta con dos estados terminales: CONFIRMADA (congelada) o ANULADA (stock devuelto)

V2__venta_confirmada.sql: 4 columnas en sales, 2 permisos (34 -> 36), CHECK recreado

Endpoints: PATCH /api/local/sales/{id}/confirm y PATCH /api/local/sales/{id}/cancel

Permisos separados: CONFIRMAR_VENTAS y CANCELAR_VENTAS (antes no existían)

Frontend: badges de estado, dialogos de confirmacion y 409 mostrado al usuario

Verificación: 24 tests nuevos, suite completa en 153 en verde

Indice

1. El bug: un numero falso escrito con un 200 OK
2. Por que Math.max era un silenciador y no una proteccion
3. La regla de negocio correcta y por que dos bucles
4. El error secundario: el costo NO se revierte
5. La pieza que faltaba: la venta no tenia estados
6. Por que dos booleanos y no un enum de estado
7. PaymentStatus y el estado de la venta: dos ejes distintos
8. Los 4 campos nuevos y la migracion V2
9. El problema clasico: el CHECK de permissions
10. Por que los permisos van en la migracion y no en el bootstrap
11. Los dos endpoints, con sus codigos HTTP
12. Las reglas de confirm() y por que es idempotente
13. Las reglas de cancel() y por que no toca el pago
14. Por que se recorren los detalles y no el request
15. Excepciones: 404, 409 y el handler global
16. El frontend: estados visibles y acciones por permiso
17. Tests: que se verifica y por que 24
18. Tabla de invariantes: el contrato resumido
19. Lo que este diseno NO resuelve (honestidad)

1. El bug: un numero falso escrito con un 200 OK

El modulo de Compras se completo en septiembre y funcionaba en las pruebas: comprar sumaba stock, guardar el costo del producto, cancelar una compra lo revertia. El flujo se veia bien porque en una compra recién hecha el stock SI alcanza para revertirla.

El problema aparecia cuando entre la compra y la cancelacion habia una VENTA. El escenario, con numeros:

El escenario que corrompia el inventario

```
comprar 10 unidades -> stock = 10
vender 5 unidades -> stock = 5 (el cliente se llevo 5 reales)
cancelar la compra -> 5 - 10 = -5
                    Math.max(-5, 0) = 0 <-- lo que se guardaba
```

Por que este bug era serio

Lo grave no es que el numero quedara raro. Es que el sistema terminaba afirmando 'no hay nada en el almacen' cuando en realidad 5 unidades ya se habian vendido, y ademas lo hacia con un **200 OK**: nadie se enteraba. Un inventario que se auto-corrige a cero es peor que uno que falla, porque deja de avisar.

La segunda mitad del problema es de vocabulario. El usuario escribio **cancelar** la compra, no **deshacer** el producto que ya se vendio. El sistema acepto una operacion que en el mundo real no existe: no se puede devolver al proveedor comida que ya comio un cliente.

2. Por que Math.max era un silenciador y no una proteccion

PurchaseImpl.java (el codigo que se elimino, con su comentario)

```
*
* <p>Desde V3 esa situación <b>no es representable</b>: mientras la compra
* está pendiente nunca tocó el inventario, así que borrarla no deja nada que
* revertir. El {@code Math.max} ya no tiene dónde aparecer porque el bug
* que lo motivaba está estructuralmente eliminado, no escondido detrás de
* una validación.
*
* <p>Una compra CONFIRMADA ya suma stock y define el costo vigente, y es
* evidencia contable de una entrega real: se rechaza con
* <b>409 CONFLICT</b> y no se borra, igual que una venta confirmada (V2).
*/
@Override
@Transactional
```

La intencion de Math.max(nuevoStock, 0) era 'el stock nunca puede ser negativo'. Y es verdad: despues de esa linea el stock nunca era negativo. Ese es exactamente el problema. Math.max no impide la operacion imposible: solo la obliga a **escribir un numero falso**.

Se creia que hacia	Realmente hacia
Evitar un error de la BD	Rellenaba silenciosamente el dato que faltaba.
Proteger al usuario	Le devolvía 200 OK sobre una operacion destruida.
Un tope de seguridad	Un tope de perdida de informacion: 5 unidades desaparecieron.

Un max(..., 0) solo es defendible cuando el cero es un valor de negocio real. Aqui no lo era: el valor correcto era **rechazar la operacion**, porque el stock de esa mercaderia ya no es de esa compra.

3. La regla de negocio correcta y por que dos bucles

La regla que se implemento

Si el stock actual de algun producto es **MENOR** que la cantidad que decia esa compra, es que esa mercaderia **ya salio** y la compra ya no se puede deshacer. Se rechaza con **409 CONFLICT** y no se escribe absolutamente nada.

PurchaseImpl.cancel() completo, tal como quedo

```

    }

    if(purchase.getDetails() != null){
        for(PurchaseDetailEntity detail : purchase.getDetails()){
            ProductEntity product = productRepository.findById(detail.getProduct().getId())
                .orElseThrow(() -> new PurchaseException(
                    "El producto asociado a la compra ya no existe; no se puede confirmar",
                    HttpStatus.CONFLICT));

            //1) Entra la mercancia al almacén.
            product.setStock(product.getStock() + detail.getQuantity());

            // !? AQUÍ ESTÁ LA DUDA FRECUENTE: se actualiza `cost` y NO `price`.
            // Son dos columnas distintas, y a propósito:
            //
            //   cost  = lo que le PAGAMOS al proveedor. Solo el módulo de
            //             compras lo escribe, porque solo él sabe lo que se
            //             pagó. Alimenta `GET /api/local/reports/margins`
            //             (margen = price - cost) y, desde V3, la utilidad.
            //   price = lo que le COBRAMOS al cliente. Es una decisión
            //             comercial (margen, competencia, promociones) y NO
            //             debe cambiar solo porque el proveedor cobró más caro.
            //
            // Si se actualizara `price` con el costo, cada compra borraría el
            // margen que el dueño eligió y el producto se vendería a precio
            // de costo. Recalcular el precio es una REGLA DE NEGOCIO
            // (p.ej. "nuevo costo + X% de margen") y va en el CRUD de
            // productos, no de paso aquí.
            product.setCost(detail.getUnitCost());
            productRepository.save(product);
        }
    }

    purchase.setConfirmed(true);
    purchase.setConfirmedAt(java.time.LocalDateTime.now());

    return PurchaseMapper.toDto(purchaseRepository.save(purchase));
}

//Validar la petición: proveedor y items obligatorios, cantidades y costos válidos (400)
private void validateRequest(PurchaseRequest request){
    if(request.getProviderId() == null){
        throw new IllegalArgumentException("El proveedor es obligatorio");
    }
    if(request.getItems() == null || request.getItems().isEmpty()){
        throw new IllegalArgumentException("La compra debe tener al menos un artículo");
    }
    for(PurchaseItemRequest item : request.getItems()){
        if(item.getProductId() == null){
            throw new IllegalArgumentException("Cada artículo debe indicar un producto");
        }
        if(item.getQuantity() == null || item.getQuantity() <= 0){
            throw new IllegalArgumentException("La cantidad debe ser mayor a 0");
        }
    }
}

```

El metodo esta partido en dos fases a proposito, y la segunda solo se ejecuta si la primera termino limpia. Los comentarios del codigo explican las tres razones por las que no se confia en el rollback de la transaccion: un mensaje util que lista todos los productos , proteccion que no depende de @Transactional (que falla si alguien invoca el metodo por auto-invocacion, sin pasar por el proxy de Spring), y legibilidad.

El esqueleto de las dos fases

```
// Fase 1: validacion -> acumula conflictos, NO escribe
if(product.getStock() < detail.getQuantity()){
    conflictos.add(String.format("'s' (comprados: %d, en almacen: %d)", ...));
}
if(!conflictos.isEmpty()){
    throw new PurchaseException("No se puede cancelar la compra: ...", CONFLICT);
}

// Fase 2: escritura -> resta directa, sin Math.max
product.setStock(product.getStock() - detail.getQuantity());
```

4. El error secundario: el costo NO se revierte

En la fase 2 solo se revierte la **cantidad**. El campo `product.cost` (ultimo costo de compra vigente) se deja como esta, y tambien `price`, el precio de venta, que no se toca nunca en el modulo de compras.

Por que el costo y el precio se dejan intactos

`cost` es el ultimo costo de compra vigente, no un promedio ni un saldo. El inventario que queda en el almacen, incluido lo que se compro en esta compra y sobrevive a su cancelacion, si costo eso. Cancelar la compra no devuelve el producto al proveedor: devuelve unidades al almacen, y las unidades que quedan valen lo que valian. El `price` es una decision comercial (el dueo decide cuanto cobra), no un dato que se deduzca del costo: por eso las compras jamas lo tocan.

5. La pieza que faltaba: la venta no tenia estados

Arreglar el `Math.max` era necesario pero no alcanzaba. Seguia habiendo una puerta abierta: si una venta se puede borrar o revertir en cualquier momento, el inventario sigue siendo manipulable despues de que el cliente se llevo la comida. Por eso la venta recibio un **ciclo de vida**, con dos estados terminales.

Estado	Que significa en el negocio	Se puede anular?	Se puede confirmar?
ABIERTA (open)	Se registro, el stock ya salio del almacen, el pedido esta en proceso.	SI, si es efectivo	SI
CONFIRMADA	El pedido ya salio del mostrador. Dinero y stock son hechos reales: CONGELADA.	NO (409)	SI, idempotente
ANULADA	Error de caja o mal cobro. El stock volvio al almacen, pero la fila se conserva.	NO (409)	NO (409)

Por que anular no es borrar

La anulacion **NO borra la venta**: la deja con `cancelled=true`. El historial de un punto de venta es evidencia contable, y un contador que baja solo es un agujero de fraude. Para el negocio la venta no conto: para la base de datos quedo registrado que paso.

Y el efecto dominó es el que cierra el bug original: si el pedido esta **confirmado** (el cliente ya se llevo la comida), entonces el stock es real, entonces la compra que lo produjo **no se puede cancelar** por la regla del punto 3. Confirmar una venta es, en la practica, la manera de que el sistema deje de permitir que se deshaga lo que ya ocurrio en el mundo real.

6. Por que dos booleanos y no un enum de estado

Un diseno tentador es un enum EstadoVenta { ABIERTA, CONFIRMADA, ANULADA } con una sola columna estado. Se descarto a proposito, por dos razones.

Como quedo en la entidad: dos flags, no un enum

```
@Entity
@Table(name = "sales")
public class SaleEntity {
    // ...
    @Column(nullable = false, columnDefinition = "boolean default false")
    private boolean confirmed;

    @Column(nullable = false, columnDefinition = "boolean default false")
    private boolean cancelled;
    private LocalDateTime confirmedAt;
    private LocalDateTime cancelledAt;
}
```

- Un enum obligaria a que confirmed y cancelled se rechazaran mutuamente en la base de datos, y la regla 'no se puede anular una confirmada' ya la valida el codigo con un mensaje de negocio claro. Un CHECK NOT (confirmed AND cancelled) seria redundante.
- Con dos booleanos, cada estado es legible por separado (se puede filtrar por 'las que no confirme todavia' con un indice, o por 'las anuladas' para una auditoria) sin tener que parsear un texto.
- Los dos boolean son primitivos de Java, no wrappers: no pueden ser null, asi que la entidad no puede quedar en un estado indeterminado por error. El NOT NULL de la BD lo refuerza.

Por que primitivos y no wrappers

```
//Un flag booleano primitivo: no puede ser null. Por eso la entidad nunca
//queda en un estado indeterminado. Con Boolean (wrapper) si podria, y habria
//que escribir un null-check en cada if.
private boolean confirmed; //primitivo
private Boolean confirmed; //wrapper: admitiria null
```

La invariante 'nunca ambas a true' la garantiza el servicio: confirm() rechaza anular, cancel() rechaza confirmar. Es una invariante de aplicacion, no de esquema, y por eso en el punto 19 se dice que es la primera debilidad conocida del diseno.

7. PaymentStatus y el estado de la venta: dos ejes distintos

Este es el punto que mas confunde al leer el codigo, asi que va con diagrama. El enum PaymentStatus (PENDING, PROCESSING, APPROVED, REJECTED, REVERSED) describe el **PAGO**: ¿se cobro?, ¿se revirtio? El estado de la venta describe el **PEDIDO**: ¿sigue abierto?, ¿ya se lo llevo el cliente? Son ejes ortogonales.

Los dos ejes, combinados

PAGO (PaymentStatus)	ESTADO DE LA VENTA (confirmed / cancelled)
APPROVED + abierta	el pago entro, el pedido sigue en proceso
APPROVED + CONFIRMADA	el flujo feliz: se cobro y se entrego
APPROVED + ANULADA	caso tarjeta: se devuelve el dinero con una REVERSA real, y la venta queda anulada
REVERSED + ANULADA	el camino de la tarjeta, completo
APPROVED + abierta + sin caja abierta	se confirmo con la caja abierta y despues se cerro el turno

La confusion que este diseno evita

Por eso el enum `PaymentStatus` **no** se toco y por eso la anulacion **no** cambia el estado del pago. Confundir los dos ejes es lo que lleva a meter un estado 'ANULADA' en `PaymentStatus` y a romper los reportes de pagos despues: un pago revertido no es lo mismo que una venta anulada.

8. Los 4 campos nuevos y la migracion V2

Los dos booleanos son `NOT NULL DEFAULT false` para que las ventas que ya existen en la base de datos (todas cobradas y entregadas hace meses) nacen como **no confirmadas**, que es el estado correcto: una venta vieja se puede confirmar hoy, y solo entonces queda congelada. Los dos timestamps quedan `NULL`: null significa 'todavia no ocurrio'.

V2_venta_confirmada.sql - los 4 ALTER TABLE

```
ALTER TABLE public.sales
  ADD COLUMN IF NOT EXISTS confirmed boolean NOT NULL DEFAULT false;
ALTER TABLE public.sales
  ADD COLUMN IF NOT EXISTS confirmed_at timestamp(6) without time zone;
ALTER TABLE public.sales
  ADD COLUMN IF NOT EXISTS cancelled boolean NOT NULL DEFAULT false;
ALTER TABLE public.sales
  ADD COLUMN IF NOT EXISTS cancelled_at timestamp(6) without time zone;
```

Campo	Tipo	Por que asi
confirmed	boolean NOT NULL DEFAULT false	Primitivo en Java; DEFAULT false para las ventas ya existentes.
confirmed_at	timestamp(6) NULL	El momento exacto del sellado. NULL mientras siga abierta.
cancelled	boolean NOT NULL DEFAULT false	Igual que confirmed, por simetria.
cancelled_at	timestamp(6) NULL	El momento de la anulacion. NULL si no se anulo.

Por que IF NOT EXISTS y por que COMMENT ON COLUMN

El `IF NOT EXISTS` en cada `ALTER` hace que la migracion sea **idempotente**: si alguien la aplico a mano, o si Hibernate dejo la columna a medias con un `ddl-auto=update` previo, volver a ejecutarla no revienta. Flyway igual no la re-ejecuta una vez registrada, pero el `IF NOT EXISTS` salva el caso de 'la corri a mano y despues la dejo escrita'. El `COMMENT ON COLUMN` deja la regla escrita en la propia base de datos, donde se lee sin abrir el codigo.

9. El problema clasico: el CHECK de permissions

Al agregar `CONFIRMAR_VENTAS` y `CANCELAR_VENTAS` al enum `PermissionName`, el arranque fallo. La razon es una de las mas desorientadoras de este proyecto y merece explicarse bien.

La tabla `permissions` tiene un `CHECK` que fija la lista EXACTA de nombres permitidos: `permissions_name_check`. Es el type-safety de la base de datos, y por eso Hibernate lo genera solo al crear la tabla. El problema: **ni Hibernate con `ddl-auto=update` ni una entidad JPA pueden ampliar un CHECK**, porque un `CHECK` es DDL puro. Asi que al insertar el permiso nuevo, la base de datos lo rechazaba con un error de violacion de restriccion.

```
V2 - DROP + ADD del CHECK, con la forma IN() en vez de =ANY(ARRAY[])
ALTER TABLE public.permissions DROP CONSTRAINT IF EXISTS permissions_name_check;

ALTER TABLE public.permissions
  ADD CONSTRAINT permissions_name_check CHECK (name IN (
    'VER_COMPRAS', 'CREAR_COMPRAS', 'CANCELAR_COMPRAS',
    ...
    'VER_VENTAS', 'CREAR_VENTAS', 'CONFIRMAR_VENTAS', 'CANCELAR_VENTAS',
    ...
    'PROCESAR_PAGOS'
  ));
```

10. Por que los permisos van en la migracion y no en el bootstrap

```
V2 - siembra de los 2 permisos, idempotente y versionada
INSERT INTO public.permissions (name)
SELECT 'CONFIRMAR_VENTAS'
WHERE NOT EXISTS (SELECT 1 FROM public.permissions WHERE name = 'CONFIRMAR_VENTAS');

--Asignarlos a los roles que los necesitan (tabla puente N:M)
INSERT INTO public.role_permissions (role_id, permission_id)
SELECT r.id, p.id
FROM public.roles r CROSS JOIN public.permissions p
WHERE r.name IN ('ADMIN','CAJERO')
  AND p.name IN ('CONFIRMAR_VENTAS','CANCELAR_VENTAS')
  AND NOT EXISTS (SELECT 1 FROM public.role_permissions rp WHERE ...);
```

Podria haberse dejado solo en el PermissionBootstrap (que inserta los faltantes al arrancar). Se puso en la migracion por una razon concreta: el RolePermissionBootstrap es 'seed inicial' a proposito, desde la decision del 17-09: si un rol base ya tiene al menos un permiso, no toca nada, para respetar la gestion manual de roles. Consecuencia directa: **en una base de datos ya sembrada, un permiso nuevo nunca llega solo a los roles** por el bootstrap.

Por que NO en el bootstrap
Poner la asignacion en la migracion la hace explicita, versionada, y que corre una sola vez (Flyway no re-ejecuta una migracion aplicada). Es la unica forma de que el permiso llegue a CAJERO y ADMIN en una base de datos que ya existia, sin que el usuario tenga que acordarse de abrir la pantalla de Roles y asignarlos a mano. El INSERT ... SELECT ... NOT EXISTS es idempotente por si acaso.

Permiso	Quien lo recibe	Que habilita
CONFIRMAR_VENTAS	ADMIN, CAJERO	Cerrar la venta en el POS (PATCH /{id}/confirm).
CANCELAR_VENTAS	ADMIN, CAJERO	Anular una venta abierta de efectivo (PATCH /{id}/cancel).

El enum de 34 a 36
El enum paso de **34 a 36 permisos**. En una base de datos ya inicializada, si solo se hubiera tocado el enum, el arranque habria fallado al insertar el permiso nuevo por el CHECK; y si se hubiera droppeado el CHECK a mano sin actualizar la migracion, el arranque de otra maquina habria fallado. V2 cierra las dos vias.

11. Los dos endpoints, con sus codigos HTTP

SaleController - los dos PATCH nuevos

```

*
* <p>Devuelve 200 con la venta actualizada. Si ya estaba confirmada también
* devuelve 200 (idempotencia); los 409 son para el caso incompatible de
* verdad, que es intentar confirmar una venta ya anulada.
*/
@PreAuthorize("hasAuthority('CONFIRMAR_VENTAS')")
@PatchMapping("/{id}/confirm")
public ResponseEntity<SaleHistoryResponse> confirm(@PathVariable Long id){
    return ResponseEntity.ok(service.confirm(id));
}

/**
* ANULAR una venta: devuelve el stock al inventario y conserva el registro
* con {@code cancelled = true}.
*
* <p>No borra la fila. En un POS el historial es evidencia contable: una
* venta anulada que desaparece deja el mismo hueco en los números que una
* venta que nunca existió, y no hay forma de auditar quién la quitó.
*
* <p>Rechazos (409) que devuelve el servicio, no el controller:
* <ul>
* <li>la venta ya está confirmada (congelada),</li>
* <li>la venta ya estaba anulada (el stock ya volvió una vez),</li>
* <li>la venta fue con tarjeta: el pago está capturado y necesita una
* reversa real vía {@code POST /api/local/payments/reverse/{id}}.</li>
* </ul>
*
* <p>Permiso {@code CANCELAR_VENTAS} para que un cajero con
* {@code CREAR_VENTAS} no pueda deshacer ventas por su cuenta.
*/
@PreAuthorize("hasAuthority('CANCELAR_VENTAS')")
@PatchMapping("/{id}/cancel")
public ResponseEntity<SaleHistoryResponse> cancel(@PathVariable Long id){
    return ResponseEntity.ok(service.cancel(id));
}

```

Se usa **PATCH** y no POST ni PUT, y no es cosmetico. PATCH significa 'cambio parcial': el cliente no reenvia la venta entera, solo pide mover su estado. Con un PUT el cliente tendria que mandar el objeto completo, y podria pisar el stock o el total mientras otro usuario edita. Ademas PUT sobre un sub-recurso (/id)/confirm) es semanticamente raro: no se esta reemplazando la venta, se esta cambiando su estado.

Endpoint	Permiso	200	404	409
PATCH /api/local/sales/{id}/confirm	CONFIRMAR_VENTAS	Venta confirmada (o ya lo estaba: idempotente)	-	La venta ya estaba anulada
PATCH /api/local/sales/{id}/cancel	CANCELAR_VENTAS	Venta anulada y stock devuelto	Venta inexistente	Ya confirmada ya anulada pago con tarjeta

Por que 409 y no 400

Un 409 CONFLICT es la respuesta correcta para 'el estado pedido no es compatible con el estado actual'. No es un 400 (no fue un parametro malo) ni un 500 (no se rompio nada). El 404 es para el id inexistente, que antes de este trabajo devolvía un 400: un recurso que no existe merece 404 aunque el cliente lo haya pedido por error.

12. Las reglas de confirm() y por que es idempotente

SaleImpl.confirm() - el congelamiento

```

* este método devolviera 400.
*/
private SaleEntity findSaleOrThrow(Long saleId){
    return saleRepository.findById(saleId)
        .orElseThrow(() -> new SaleException(
            "Venta no encontrada con ID: " + saleId, HttpStatus.NOT_FOUND));
}

/**
 * CONFIRMAR la venta: la congela definitivamente.
 *
 * <p>Se ejecuta cuando el pedido ya salió del mostrador. Desde este punto el
 * stock descontado y el dinero cobrado son hechos reales, así que la venta
 * ya no se puede anular ni borrar. Es lo que hace imposible, en cascada, que
 * alguien intente "deshacer" una COMPRA cuyo stock ya se vendió.
 *
 * <p><b>Idempotente por diseño</b>: si ya estaba confirmada devuelve OK sin
 * volver a pisar la fecha. La UI puede sufrir doble clic y el usuario puede
 * reintentar tras un corte de red; un 409 ahí sería confuso ("¿ya la
 * confirmé o no?"). Confirmar dos veces no es un error, es el mismo estado
 * pedido dos veces.
 */
@Override
@Transactional
public SaleHistoryResponse confirm(Long saleId){
    SaleEntity sale = findSaleOrThrow(saleId);

    //Una venta anulada es terminal en la otra dirección: no se "revive".
    if(sale.isCancelled()){
        throw new SaleException(
            "No se puede confirmar una venta que ya fue anulada", HttpStatus.CONFLICT);
    }

    if(sale.isConfirmed()){
        //Ya está en el estado pedido: no es un error, es idempotencia.
        log.info("Venta {} ya estaba confirmada: operación idempotente.", saleId);
        return mapper.toHistoryResponse(sale);
    }
}

```

- **Si no existe -> 404** (via findSaleOrThrow, un helper que confirm() y cancel() comparten).
- **Si ya esta anulada -> 409**: una venta anulada es terminal en la otra direccion, no se revive. Anular y luego confirmar seria resucitar una venta.
- **Si ya esta confirmada -> 200, sin cambios**: esto es idempotencia a proposito. La UI puede sufrir doble clic y el usuario puede reintentar tras un corte de red; un 409 ahí seria confuso ('¿ya la confirme o no?'). Confirmar dos veces no es un error, es el mismo estado pedido dos veces.
- **Si esta abierta -> la confirma** y le pone el timestamp.

Por que confirmar es idempotente

Idempotente significa que hacer la operacion N veces tiene el mismo efecto que hacerla una. Es la propiedad que hace segura una API expuesta a clics dobles, reintentos y pestanas abiertas. Frente a la alternativa de devolver 409 (que obliga al frontend a distinguir 'ya estaba' de 'me la rechaza'), el 200 con el estado actual deja que el frontend siempre pueda pintar lo que hay.

13. Las reglas de cancel() y por que no toca el pago

SaleImpl.cancel() - la devolución de stock

```

*      la única fuente de verdad.</li>
* </ul>
*
* <p>Ojo con la devolución: se recorre {@code sale.getDetails()} (los
* renglones ya guardados) y NO los items del request, porque en una
* anulación no hay request: hay que deshacer exactamente lo que se hizo.
* La cantidad a devolver es la del HISTÓRICO, no la que el cliente mande.
*/
@Override
@Transactional
public SaleHistoryResponse cancel(Long saleId){
    SaleEntity sale = findSaleOrThrow(saleId);

    if(sale.isConfirmed()){
        throw new SaleException(
            "No se puede anular una venta ya confirmada: el pedido ya salió y el stock es real",
            HttpStatus.CONFLICT);
    }

    if(sale.isCancelled()){
        throw new SaleException("La venta ya estaba anulada", HttpStatus.CONFLICT);
    }

    if(sale.getPaymentMethod() != PaymentMethod.CASH){
        throw new SaleException(
            "Una venta con tarjeta no se anula desde aquí: el pago fue capturado y necesita una reversa real "
            + "en el módulo de pagos", HttpStatus.CONFLICT);
    }

    //Devolver el stock renglón por renglón.
    if(sale.getDetails() != null){
        for(SaleDetailEntity detail : sale.getDetails()){
            ProductEntity product = productRepository.findById(detail.getProduct().getId())
                .orElseThrow(() -> new SaleException(
                    "El producto " + detail.getProduct().getName()
                    + " ya no existe; no se puede anular la venta",
                    HttpStatus.CONFLICT));

            //SUMA de vuelta. Sin Math.max aquí a propósito: en una anulación
            // legítima el stock siempre alcanza (o queda en 0) porque
            // validateStock() en processSale ya lo garantizó. Si no
            // alcanzara, el dato de la BD está corrupto, y enmascararlo con
            // max(...,0) solo escondería el problema.
            product.setStock(product.getStock() + detail.getQuantity());
            productRepository.save(product);
        }
    }
}

```

Las reglas, en orden en que se evalúan:

- **Confirmada -> 409.** Es el congelamiento: el cliente ya se llevó la comida.
- **Ya anulada -> 409.** Si pasara, el stock volvería a subir dos veces y el inventario quedaría inflado. Es el gemelo del bug de inventario, pero del lado de la anulación.
- **Pago con tarjeta -> 409.** El dinero ya entro a la cuenta; 'anular la venta' aquí no devolvería nada, solo mentiría el historial. Para ese caso existe el flujo de reversa real (POST /api/local/payments/reverse/{id}), que además exige su propia autorización. Una venta en efectivo sí se puede anular, porque el efectivo se devuelve en mano y el estado de la venta es la única fuente de verdad.

Por que la tarjeta no se anula desde aqui

Que solo el efectivo se pueda anular desde aqui es una decision, no una limitacion de tiempo. Si se permitiera anular una venta de tarjeta 'para que el cajero lo arreglara', el historial mostraria una venta anulada cuyo pago sigue APPROVED: dos verdades incompatibles en la misma fila. Con la tarjeta, el dinero se devuelve por el modulo de pagos, que sabe que transaccion revertir.

14. Por que se recorren los detalles y no el request

La regla de oro: anular no recibe datos

La devolucion de stock recorre `sale.getDetails()` (los renglones ya guardados) y NO los items del request, porque en una anulacion no hay request: hay que deshacer **exactamente lo que se hizo**. La cantidad a devolver es la del HISTORICO, no la que el cliente mande. Si se aceptara un body con cantidades, un cliente podria inventar una cantidad distinta y al anular moveria el stock incorrecto.

Por eso el endpoint no declara ningun `@RequestBody`: no hay nada que mandar. El unico dato que identifica la venta es el id de la ruta. Es la firma de una operacion que no acepta influencia externa.

Por que el cliente manda un {} si no hay cuerpo

El cliente si envia {} como cuerpo, y no es descuido: la firma de `HttpClient.patch(url, body)` en Angular exige el segundo argumento. Se manda un objeto vacio para cumplir la firma, y el backend lo ignora porque no hay `@RequestBody` que lo lea. La convencion del proyecto es enviar {} en lugar de null: es la convencion del proyecto, para no andar pasando null por el codigo.

15. Excepciones: 404, 409 y el handler global

```
SaleException - una excepcion que transporta su propio codigo HTTP
/**
 * Excepción de negocio del módulo de ventas.
 *
 * <p>Por defecto 400 BAD_REQUEST, que es lo que devolvía antes de agregarle el
 * HttpStatus: stock insuficiente, caja no abierta al vender, método de pago
 * inválido, venta sin productos, etc. son todos "pedí algo que no se puede
 * cumplir tal cual", no 404.
 *
 * <p>El segundo constructor permite que una misma excepción represente otros
 * códigos cuando la REGLA lo pida. Se usa en el ciclo de vida de la venta
 * (2026-09-30) para devolver 409 CONFLICT en los casos donde el recurso existe
 * pero la operación choca con su estado:
 * <ul>
 *   <li>anular una venta ya confirmada (congelada),</li>
 *   <li>anular una venta ya anulada (devolvería el stock dos veces),</li>
 *   <li>anular una venta con tarjeta (el pago fue capturado: necesita reversa),</li>
 *   <li>confirmar una venta ya anulada.</li>
 * </ul>
 *
 * <p>El patrón es idéntico al de {@code ProductException} / {@code ProviderException}
 * / {@code PurchaseException}: el servicio lanza la excepción con el código
 * desired y {@code GlobalExceptionHandler} solo la traduce. Así el servicio no
 * necesita saber de HTTP y el handler no necesita saber de ventas.
 */
public class SaleException extends RuntimeException {
    private final HttpStatus status;

    /** 400 BAD_REQUEST: el comportamiento por defecto histórico. */
    public SaleException(String message) {
        this(message, HttpStatus.BAD_REQUEST);
    }

    /** Código explícito, p. ej. HttpStatus.CONFLICT (409) o NOT_FOUND (404). */
    public SaleException(String message, HttpStatus status) {
        super(message);
        this.status = status;
    }

    public SaleException(String message, Throwable cause) {
        super(message, cause);
        this.status = HttpStatus.BAD_REQUEST;
    }

    /**
     * Estado HTTP que debe devolverse al cliente (lo usa GlobalExceptionHandler).
     */
    public HttpStatus getStatus() {
        return status;
    }
}
```

Antes, SaleException era solo un RuntimeException y el handler global la traducía a 400 siempre. Ahora lleva un HttpStatus: el servicio dice que error es, y el handler lo traduce. Así el 404 (no existe) y el 409 (estado incompatible) salen del servicio, no de un if-else en el handler.

Donce se lanza	Excepcion	Status
findSaleOrThrow (id inexistente)	SaleException	404 NOT_FOUND
confirm (venta anulada)	SaleException	409 CONFLICT
cancel (venta confirmada / ya anulada / tarjeta)	SaleException	409 CONFLICT

Donde se lanza	Excepcion	Status
validateRequest / validateStock / pago insuficiente	SaleException	400 (default)

El handler global, un solo lugar

Con esto, cualquier excepcion de negocio que se escape del servicio produce el mismo formato JSON { code, message } con el codigo HTTP correcto, sin un try/catch en cada metodo del controller. Es el @RestControllerAdvice global haciendo su trabajo.

16. El frontend: estados visibles y acciones por permiso

El backend sin UI deja el trabajo a medias: el usuario no sabe que puede confirmar ni que no puede anular. La pantalla de historial de ventas (Angular) recibio badges de estado, los botones de acción, y una regla de que solo se muestra 'Anular' en efectivo.

Compras-Frontend-Local - la interfaz espeja los campos nuevos

```
//Interface con atributos que se usarán en la interface SaleRequest
export interface SaleItemRequest {
  productId: number;
  quantity: number;
}

//Interface para ventas que se usará en sale-service
export interface SaleRequest {
  paymentMethod: PaymentMethod;
  cashReceived?: number;
  items: SaleItemRequest[]; //Un arreglo con los atributos de la interface SaleItemRequest
  cardPayment?: CardPaymentRequest; //Para pagos con tarjetas
}

//Interface para ventas que se usará en sale-service
export interface SaleResponse {
  saleId: number;
  total: number;
  paymentMethod: PaymentMethod;
  changeAmount: number | null;
  cashReceived: number | null;
  paymentStatus?: 'PENDING' | 'APPROVED' | 'REJECTED';
  cardPaymentResponse?: CardPaymentResponse;
}

//Interface para el historial de la venta
export interface SaleHistory {
  id: number;
  saleDate: string;
  total: number;
  paymentMethod: PaymentMethod;
  cashReceived: number | null;
  changeAmount: number | null;

  // ===== Ciclo de vida de la venta (2026-09-30) =====
  // El backend expone las DOS banderas y no un 'estado' derivado. Se replica
  // el contrato tal cual, a propósito: si aquí se derivara un string 'ABIERTA'
  // / 'CONFIRMADA' / 'ANULADA', estaríamos reimplementando la máquina de
  // estados del servidor en el cliente, y en cuanto el backend agregara un
  // estado nuevo el frontend mostraría algo equivocado sin avisar.
  //
  // Los 3 estados válidos y qué botón habilita cada uno:
  //   confirmed=false, cancelled=false -> abierta: se puede confirmar o anular
  //   confirmed=true                    -> congelada: ningún botón
  //   cancelled=true                    -> anulada: el stock ya volvió
  confirmed: boolean;
  confirmedAt: string | null;
  cancelled: boolean;
  cancelledAt: string | null;
}
```

sale-service.ts - los dos PATCH y el comentario que explica por que no se captura el error

```
//Ver historial de las ventas
getSales(){
  return this.http.get<SaleHistory[]>(this.api);
}

// =====
// Ciclo de vida de la venta (2026-09-30)
// =====
/*
 * Las tres observaciones de este bloque:
 *
 * 1) Se usa PATCH, no POST ni PUT. La venta YA existe; lo único que se pide
 *    es mover su estado. POST crearía algo nuevo, y PUT reenviaría la venta
 *    entera, con el riesgo de pisar campos que otro usuario cambió.
 *
 * 2) El id va en la URL y no en un body. Por eso no hay parámetro `body` en la
 *    llamada: el servidor no espera cuerpo en estos dos endpoints.
 *
 * 3) NO se captura el error aquí. El backend responde 409 con un mensaje
 *    {'code': 'SALE_ERROR', 'message': '...'} que es el texto exacto que
 *    necesita ver el cajero ("no se puede anular una venta ya confirmada").
 *    Si el servicio lo transformara, se perdería ese detalle y el usuario
 *    vería un "algo falló" inútil. Lo maneja el componente, con el mismo
 *    patrón alert(err.error?.message) que ya usa el módulo de compras.
 */

//Confirmar (congelar) una venta: ya no se puede anular ni borrar
confirmSale(id: number): Observable<SaleHistory>{
  return this.http.patch<SaleHistory>(`${this.api}/${id}/confirm`, {});
}

//Anular una venta: devuelve el stock. Solo efectivo y no confirmadas
cancelSale(id: number): Observable<SaleHistory>{
  return this.http.patch<SaleHistory>(`${this.api}/${id}/cancel`, {});
}
```

El servicio deja pasar el error sin transformarlo, a proposito: el backend responde 409 con { code: 'SALE_ERROR', message: '...' }, y ese texto es exactamente lo que necesita ver el cajero. Si el servicio lo tradujera a un 'algo fallo', se perdería el motivo. Lo maneja el componente, con el mismo patron alert(err.error?.message) que ya usa el modulo de compras.

- **Badges de estado** en la columna 'Estado': 'Pendiente' (abierta), 'Confirmada' (congelada), 'Anulada' (tachada). La fila anulada se dibuja tachada y atenuada.
- **Confirmar** sale con un dialogo de confirmacion y llama al PATCH /confirm. El boton solo se pinta si el usuario tiene CONFIRMAR_VENTAS.
- **Anular** pide confirmacion y llama al PATCH /cancel, solo si el usuario tiene CANCELAR_VENTAS y la venta es de efectivo y esta abierta.
- **El 409 se muestra al usuario** con un alert(err.error?.message): el backend explica por que (ya confirmada, ya anulada, tarjeta) y la UI no traga el error en silencio.
- Tras una accion, la fila del arreglo se reemplaza por la respuesta del backend, que ya trae los timestamps, para que el estado pintado sea el del servidor y no una optimista local.

Compras-Frontend-Local - la regla del boton Anular

```
//Ejemplo de la regla: solo efectivo, y con el permiso
canCancel(sale: SaleHistory): boolean {
  return !sale.confirmed && !sale.cancelled
    && sale.paymentMethod === 'CASH'
    && this.auth.hasPermission('CANCELAR_VENTAS');
}
```

La UI sugiere; el servidor decide

La UI no es la que garantiza la regla: el backend la vuelve a validar. Si alguien llama al PATCH /cancel desde la consola del navegador sobre una venta de tarjeta, el servicio responde 409 igual. La UI esconde el boton para no ofrecer un camino que va a fallar, pero la decision es del servidor.

17. Tests: que se verifica y por que 24

La suite anterior tenia 129 tests; este trabajo agrego **24** en tres archivos, todos en verde, para 153 en total. Un bug de inventario se blinde con ejemplos concretos del escenario, no con un 'test de que no revienta'.

Archivo	Tests nuevos	Que fijan
PurchaseImplTest	4	stock insuficiente bloquea (409); stock en 0 bloquea; dos renglones, uno conflictivo, no escribe ninguno; producto borrado -> 409.
SaleControllerTest	8	Los dos PATCH con su permiso y sin el (403); 404 de id inexistente; 409 de confirmar-anulada y anular-confirmada; anular-tarjeta; idempotencia de confirmar.
SaleImplLifecycleTest	11 (nuevo)	confirmar marca flags y timestamp; confirmar dos veces es idempotente; anular devuelve el stock; anular dos veces es 409; anular tarjeta es 409; anular sin detalles no rompe; 404 de venta inexistente; el mapper expone los flags.

PurchaseImplTest - el invariante, en codigo

```
//El caso que importa: stock = 5, se compro 10, la compra NO se cancela
when(purchaseRepository.findDetailedById(1L)).thenReturn(Optional.of(compraDe10));
when(productRepository.findById(1L)).thenReturn(Optional.of(productoConStock5));
```

```
PurchaseException ex = assertThrows(PurchaseException.class,
    () -> service.cancel(1L));
assertEquals(HttpStatus.CONFLICT, ex.getStatus());
verify(productRepository, never()).save(any()); //<-- NADIE fue escrito
```

El assert que blinda el bug original

La linea mas importante de todos los tests es `verify(productRepository, never()).save(any())`. No basta con que la excepcion se lance: hay que verificar que **no se escribio nada**, porque la promesa del error es 'no toco el inventario', no solo 'me avise'. Un test que solo mira el 409 pasaria aun con el `Math.max` adentro.

Como se corre la suite completa

```
.\mvnw.cmd -o clean test "-Dtest=!ComprasApplicationTests" \
    "-DfailIfNoSpecifiedTests=false"
# Tests run: 153, Failures: 0, Errors: 0, Skipped: 0
```

18. Tabla de invariantes: el contrato resumido

Si solo se lee una pagina de este PDF, que sea esta. Cada fila es una promesa que el codigo y los tests sostienen.

Invariante	Comportamiento	Donde se aplica
Anular dos veces la misma venta	409, y el stock NO sube dos veces	<code>cancel()</code> revisa <code>isCancelled()</code> antes de devolver stock
Confirmar dos veces la misma venta	200 idempotente, el timestamp no se pisa	<code>confirm()</code> revisa <code>isConfirmed()</code> y devuelve el estado
Anular una venta confirmada	409 (congelada)	<code>cancel()</code> revisa <code>isConfirmed()</code>
Confirmar una venta anulada	409 (no se revive)	<code>confirm()</code> revisa <code>isCancelled()</code>

Invariante	Comportamiento	Donde se aplica
Anular una venta de tarjeta	409, se usa la reversa real del modulo de pagos	cancel() exige PaymentMethod.CASH
Devolver el stock de un producto borrado	409, la venta no se toca	findById().orElseThrow en el bucle
Cancelar una compra cuyo stock se vendio	409, con la lista de productos afectados	Fase 1 de PurchaseImpl.cancel() acumula conflictos
Cancelar una compra con 2 renglones, 1 en conflicto	409, y NINGUN stock se escribe	Fase 2 solo corre si la fase 1 termino limpia
El costo del producto al cancelar una compra	No se revierte (es el ultimo costo vigente)	Solo se resta la cantidad
El precio de venta al comprar	No se toca (es decision comercial)	PurchaseImpl nunca escribe price

19. Lo que este diseno NO resuelve (honestidad)

Un documento que solo lista lo que quedo bien no sirve para repetir el trabajo. Estas son las debilidades conocidas de este diseno, para que la proxima iteracion no las descubra por sorpresa:

- **La exclusion mutua vive en el codigo, no en la BD.** confirmed y cancelled nunca deben ser ambos true, y lo garantiza SaleImpl, pero no hay un CHECK NOT (confirmed AND cancelled). Un UPDATE manual en la base de datos podria dejar una venta en un estado imposible. El CHECK se puede agregar en una V3.
- **confirm() y cancel() no usan bloqueo pesimista.** La exclusion mutua se evalua con un if sobre la entidad leida, no con un SELECT ... FOR UPDATE. Dos peticiones simultaneas podrian pasar el mismo if a la vez (un doble clic con dos pestanas) y, en el caso de cancel(), devolver el stock dos veces. El riesgo es real pero estrecho: es la misma transaccion de MySQL/Postgres la que hace el UPDATE, y el caso de uso real (un solo cajero) lo hace improbable. Arreglo futuro: @Lock(LockModeType.PESSIMISTIC_WRITE) en findById, o un UPDATE condicional WHERE cancelled = false.
- **La regla de la compra es una heuristica de stock, no un historico.** Se rechaza cancelar una compra si el stock actual es menor que lo comprado, que es correcto en el caso comun. Pero si entre la compra y la cancelacion hubo otra compra que repuso el mismo producto, el stock podria volver a alcanzar la cifra y la cancelacion pasaria. Es un escenario de dos compras del mismo producto cuyo stock se vendio entre medio: la regla lo dejaria pasar. Un historico de movimientos (quien consumo que) lo resolveria, pero es una tabla nueva para un caso poco frecuente.